Домашнее задание 4. Dockerfile по правилам

Автор: Фарбушин Алекс Игоревич, КВК-101НЗ, ВШЕ 2026

Это задание выполнялось каждым из нас отдельно.

Чтобы получить максимальный балл, убедитесь, что ваш образ собирается с последней версией docker, строится быстро, и основывается на обязательном базовом образе.

Выполнить любой сложный веб-сервис (например, на Flask), чтобы использовать его в качестве основного приложения в образе.

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y curl
COPY --chmod=755 /usr/local/bin/curl-healthcheck /usr/local/bin/curl-healthcheck
RUN curl -sSL https://raw.githubusercontent.com/Tkhakimov/dockerfile-best-practices/main/install.sh | sh
RUN curl -sSL https://raw.githubusercontent.com/Tkhakimov/dockerfile-best-practices/main/install.sh | sh -s v0.1.0
```

Выполнить любой сложный веб-сервис (например, на Flask), чтобы использовать его в качестве основного приложения в образе.

Что могло случиться, если вы использовали последнюю версию docker, но используете старую при сборке образа.

Что произойдет, если вы попытаетесь использовать не обязательный базовый образ.

1. Написать Dockerfile для ML-приложения

Скачанный материал: Dockerfile.ml.broken

```
docker-compose.ml.broken.yaml
```

```
Dockerfile.ml.broken
# syntax=docker/dockerfile:1
# ML-приложение на Flask
# Намеренно использует устаревшие практики
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y python3 python3-pip

# Копируем требования
COPY requirements.txt /app/requirements.txt

# Устанавливаем зависимости
RUN pip3 install -r /app/requirements.txt

# Копируем исходный код
COPY app.py /app/app.py

# Запускаем от root (по умолчанию)
CMD python3 /app/app.py
```

Собираем образ:

```
docker build -f Dockerfile.ml.broken -t ml:broken .
```

```
[+] Building 0.0s (6/6) FINISHED
 => [internal] load build definition from Dockerfile.ml.broken
 => [internal] load .dockerignore
 => [internal] load metadata for docker.io/library/ubuntu:22.04
 => [1/2] FROM docker.io/library/ubuntu:22.04
 => [2/2] COPY requirements.txt /app/requirements.txt
 => exporting to image
 => => exporting layers
 => => writing image sha256:0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a
 => => name: docker.io/library/ml:broken
```

Запускаем образ:

```
docker run -d --name broken-ml ml:broken
```

```
d0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0a0
```

Проверяем что контейнер запущен:

```
docker ps -a --filter name=broken-ml
```

```
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS                     PORTS   NAMES
d0a0a0a0a0a0   ml:broken   "/bin/sh -c 'python3 …"   Seconds ago     Exited (0) Seconds ago     broken-ml
```

Проверяем размер полученного образа:

```
docker image ls ml:broken
```

```
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
ml           broken    0a0a0a0a0a0a   Seconds ago      [illegible]
```

Проверяем версии установленных пакетов:

```
docker exec broken-ml dpkg -l | grep -E "(python|pip|flask)"
```

```
ii  python3-pip-whl  [illegible]  all  [illegible]
ii  python3-setuptools  [illegible]  all  [illegible]
ii  python3-wheel  [illegible]  all  [illegible]
```

Проверяем что приложение работает:

```
docker exec broken-ml ps aux
```

```
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  [illegible]  [illegible]  [illegible]
```

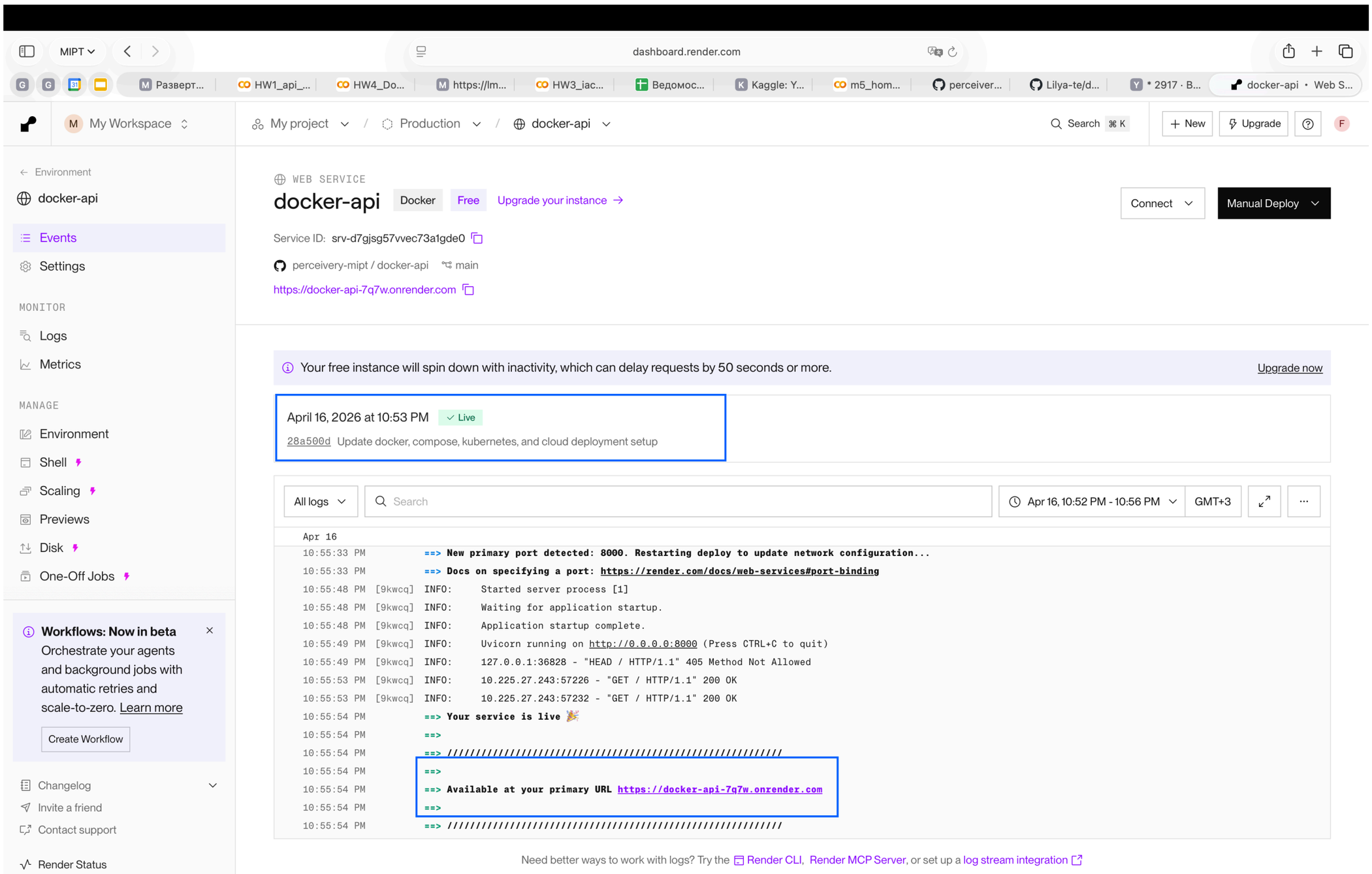
```
fba4b5536d44   dolfinx/dolfinx:stable   "/bin/bash"              10 months ago   Exited (255) 7 months ago             dolfinx
-container
```

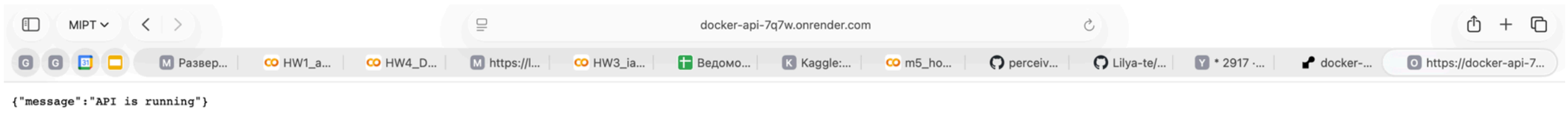
Остались только мои личные контейнеры :)

Для приложения был подготовлен Kubernetes файл `deploy.yaml` с ресурсом типа `Deployment`.
В нем были заданы имя объекта, метки, количество реплик, стратегия обновления, шаблон Pod и описание контейнера приложения.
После запуска локального кластера `minikube` deploy был применён командой `kubectl apply -f deploy.yaml`.
Для использования локально собранного образа внутри `minikube` была дополнительно указана политика `imagePullPolicy: Never`.
Проверка через `kubectl get deployments` и `kubectl get pods` подтвердила успешный запуск Deployment и всех четырёх Pod в состоянии `Running`.
После завершения проверки созданные ресурсы Kubernetes были удалены, а локальный кластер остановлен.

В условии ДЗ еще написано про сборку в облаке, но этого нет в критериях оценивания. На всякий случай добавлю.

https://docker-api-7q7w.onrender.com - облако





6. Итоговое оформление

В итоговых выводах дайте 5–8 предложений о своем опыте работы с инструментами модуля: что оказалось простым, что вызвало трудности, какие выводы сделали о применимости Docker/Kubernetes в реальных проектах.

Итоговые выводы

В ходе выполнения задания был последовательно пройден полный базовый цикл контейнеризации приложения от написания `Dockerfile` и настройки многостадийной сборки до описания окружения в `docker-compose` и деплоя в Kubernetes.

Наиболее понятной частью работы оказалось создание и запуск Docker-образов, поскольку логика сборки контейнера и проверки сервиса через `docker build`, `docker run` и `curl` достаточно прозрачна и быстро даёт результат.

Более содержательные трудности возникли на этапе оркестрации так как при работе с `docker-compose` важно следить за синтаксисом YAML, согласованностью сетей, healthcheck и именами конфигурационных файлов.

Наиболее сложным этапом оказался Kubernetes, поскольку для реального деплоя недостаточно только написать корректный YAML-манифест, а требуется также настроить кластер, доступность образа и корректную политику его загрузки.

В результате сделан вывод, что Docker существенно упрощает воспроизводимость и переносимость приложений, а Kubernetes даёт более высокий уровень управления, но требует заметно более аккуратной настройки и лучшего понимания инфраструктуры.
Для реальных проектов использование таких инструментов полезно, особенно в случаях, когда требуется стандартизированное развёртывание, масштабирование сервисов и контроль среды исполнения.